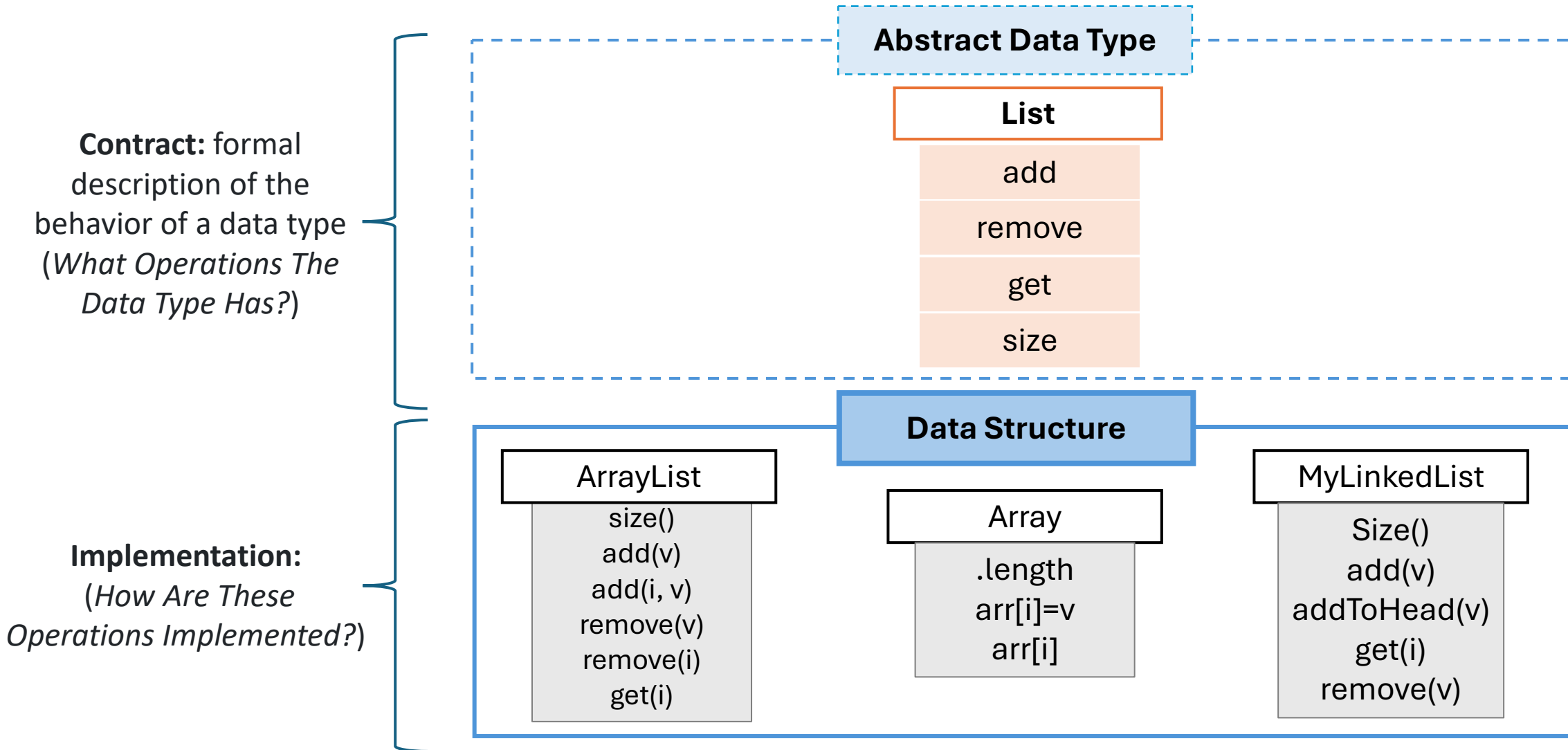


# Stacks & Queues

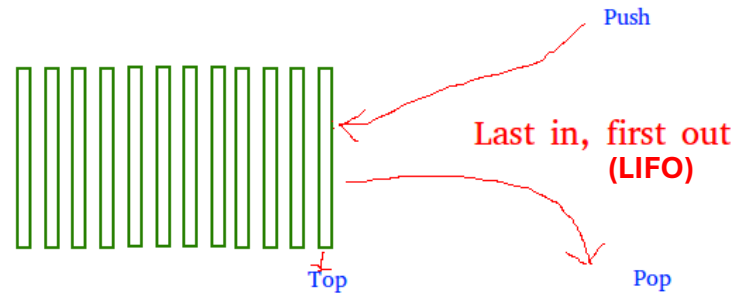
# Abstract Data Type & Data Structures



# Stack vs Queue

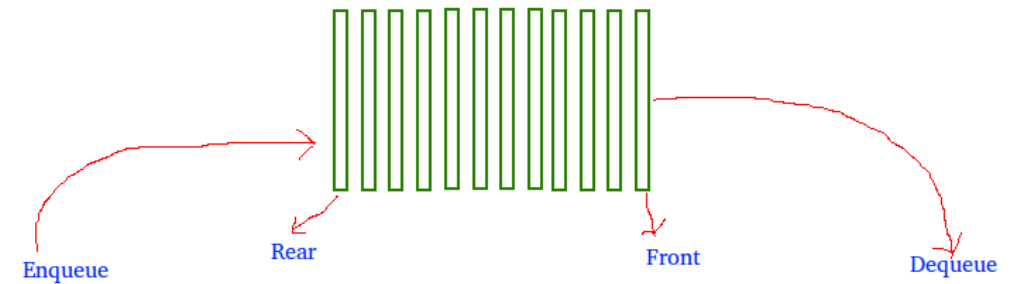
## Stack

Insertion and Deletion happen on same end

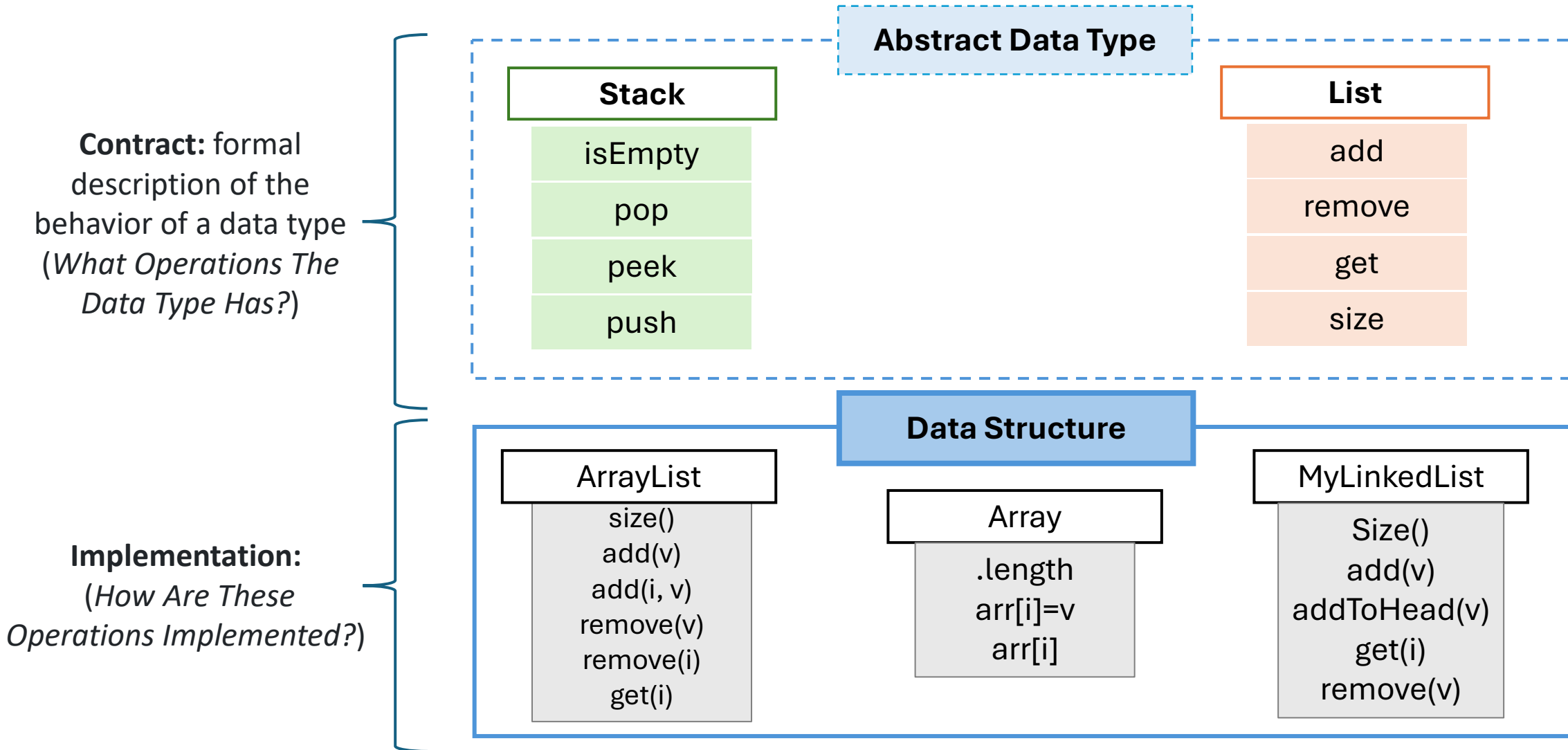


## Queue

Insertion and Deletion happen on different ends



# Abstract Data Type & Data Structures



# ArrayList & MyLinkedList Methods

## ArrayList<E>

| Method                                 | Parameters                    | Return Type                        | Description                                 |
|----------------------------------------|-------------------------------|------------------------------------|---------------------------------------------|
| <code>add(E e)</code>                  | element <code>E</code>        | <code>boolean</code>               | Appends element to end                      |
| <code>add(int index, E e)</code>       | index, element <code>E</code> | <code>void</code>                  | Inserts at position                         |
| <code>remove(int index)</code>         | index                         | <code>E</code>                     | Removes and returns element at index        |
| <code>remove(Object o)</code>          | object <code>o</code>         | <code>boolean</code>               | Removes first occurrence, returns success   |
| <code>clear()</code>                   | –                             | <code>void</code>                  | Removes all elements                        |
| <code>get(int index)</code>            | index                         | <code>E</code>                     | Returns element at index                    |
| <code>set(int index, E e)</code>       | index, element <code>E</code> | <code>E</code>                     | Replaces and returns old element            |
| <code>contains(Object o)</code>        | object <code>o</code>         | <code>boolean</code>               | Checks if element exists                    |
| <code>indexOf(Object o)</code>         | object <code>o</code>         | <code>int</code>                   | First index or <code>-1</code> if not found |
| <code>lastIndexOf(Object o)</code>     | object <code>o</code>         | <code>int</code>                   | Last index or <code>-1</code> if not found  |
| <code>size()</code>                    | –                             | <code>int</code>                   | Number of elements                          |
| <code>isEmpty()</code>                 | –                             | <code>boolean</code>               | <code>true</code> if list is empty          |
| <code>toArray()</code>                 | –                             | <code>Object[]</code>              | Returns all elements as array               |
| <code>toArray(T[] a)</code>            | array of type <code>T</code>  | <code>T[]</code>                   | Returns typed array                         |
| <code>iterator()</code>                | –                             | <code>Iterator&lt;E&gt;</code>     | Forward traversal                           |
| <code>listIterator()</code>            | –                             | <code>ListIterator&lt;E&gt;</code> | Bidirectional traversal                     |
| <code>listIterator(int index)</code>   | index                         | <code>ListIterator&lt;E&gt;</code> | Starts at given position                    |
| <code>subList(int from, int to)</code> | start, end indices            | <code>List&lt;E&gt;</code>         | View of portion of list                     |

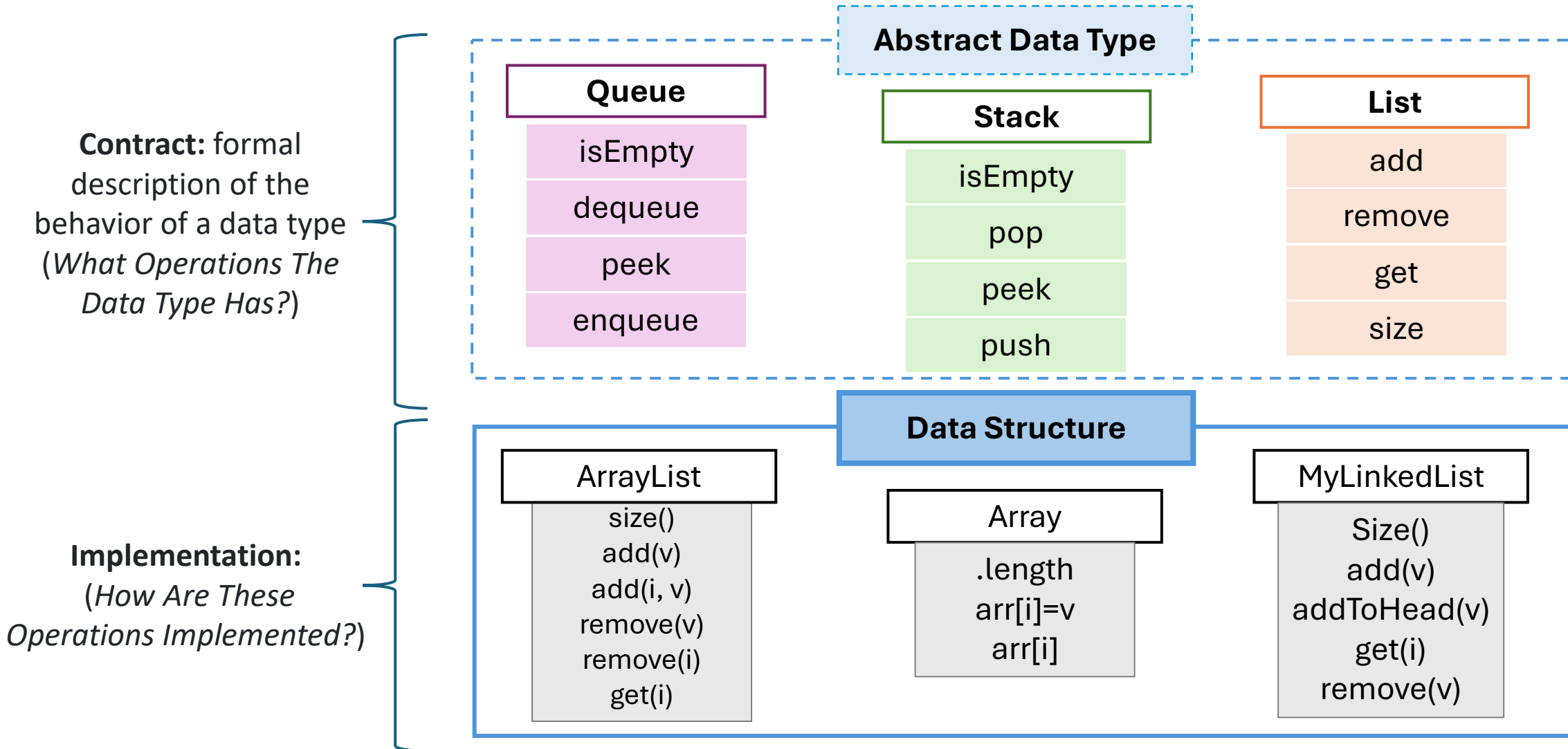
## MyLinkedList<ListType>

- `head : Node<ListType>`
- `size : int`
- `MyLinkedList()`
- `addToHead(ListType) : void`
- `add(ListType) : void`
- `badSize() : int`
- `size() : int`
- `get(int) : ListType`
- `contains(ListType) : boolean`
- `indexOf(ListType) : int`
- `remove(ListType) : boolean`
- `removeAll(ListType) : void`
- `toString() : String`

# Stack Complexity Analysis

| Data Structure     | Push   | Pop    | Peek   | Notes                    |
|--------------------|--------|--------|--------|--------------------------|
| Stack (ArrayList)  | $O(n)$ | $O(1)$ | $O(1)$ | Resize makes push $O(n)$ |
| Stack (LinkedList) | $O(1)$ | $O(1)$ | $O(1)$ | All worst-case $O(1)$    |

# Abstract Data Type & Data Structures



# ArrayList & MyLinkedList Methods

## ArrayList<E>

| Method                                 | Parameters                    | Return Type                        | Description                                 |
|----------------------------------------|-------------------------------|------------------------------------|---------------------------------------------|
| <code>add(E e)</code>                  | element <code>E</code>        | <code>boolean</code>               | Appends element to end                      |
| <code>add(int index, E e)</code>       | index, element <code>E</code> | <code>void</code>                  | Inserts at position                         |
| <code>remove(int index)</code>         | index                         | <code>E</code>                     | Removes and returns element at index        |
| <code>remove(Object o)</code>          | object <code>o</code>         | <code>boolean</code>               | Removes first occurrence, returns success   |
| <code>clear()</code>                   | -                             | <code>void</code>                  | Removes all elements                        |
| <code>get(int index)</code>            | index                         | <code>E</code>                     | Returns element at index                    |
| <code>set(int index, E e)</code>       | index, element <code>E</code> | <code>E</code>                     | Replaces and returns old element            |
| <code>contains(Object o)</code>        | object <code>o</code>         | <code>boolean</code>               | Checks if element exists                    |
| <code>indexOf(Object o)</code>         | object <code>o</code>         | <code>int</code>                   | First index or <code>-1</code> if not found |
| <code>lastIndexOf(Object o)</code>     | object <code>o</code>         | <code>int</code>                   | Last index or <code>-1</code> if not found  |
| <code>size()</code>                    | -                             | <code>int</code>                   | Number of elements                          |
| <code>isEmpty()</code>                 | -                             | <code>boolean</code>               | <code>true</code> if list is empty          |
| <code>toArray()</code>                 | -                             | <code>Object[]</code>              | Returns all elements as array               |
| <code>toArray(T[] a)</code>            | array of type <code>T</code>  | <code>T[]</code>                   | Returns typed array                         |
| <code>iterator()</code>                | -                             | <code>Iterator&lt;E&gt;</code>     | Forward traversal                           |
| <code>listIterator()</code>            | -                             | <code>ListIterator&lt;E&gt;</code> | Bidirectional traversal                     |
| <code>listIterator(int index)</code>   | index                         | <code>ListIterator&lt;E&gt;</code> | Starts at given position                    |
| <code>subList(int from, int to)</code> | start, end indices            | <code>List&lt;E&gt;</code>         | View of portion of list                     |

## MyLinkedList<ListType>

- `head : Node<ListType>`
- `size : int`
- <sup>c</sup> `MyLinkedList()`
- `addToHead(ListType) : void`
- <sup>▲</sup> `add(ListType) : void`
- `badSize() : int`
- <sup>▲</sup> `size() : int`
- <sup>▲</sup> `get(int) : ListType`
- `contains(ListType) : boolean`
- `indexOf(ListType) : int`
- `removeHead() : ListType`
- <sup>▲</sup> `remove(ListType) : boolean`
- `removeAll(ListType) : void`
- <sup>▲</sup> `toString() : String`

# Stack & Queue Complexity Analysis

| Data Structure     | Push/Enqueue | Pop/Dequeue | Peek   | Notes                      |
|--------------------|--------------|-------------|--------|----------------------------|
| Stack (ArrayList)  | $O(n)$       | $O(1)$      | $O(1)$ | Resize makes push $O(n)$   |
| Stack (LinkedList) | $O(1)$       | $O(1)$      | $O(1)$ | All worst-case $O(1)$      |
| Queue (ArrayList)  | $O(n)$       | $O(n)$      | $O(1)$ | Bad unless circular buffer |
| Queue (LinkedList) | $O(1)$       | $O(1)$      | $O(1)$ | All worst-case $O(1)$      |